# INTERNSHIP PROJECT REPORT

## Task 2: Network Traffic Packet Inspection & Analysis with Wireshark

**Domain: Cyber Security | Technical Track – Intermediate**

**Prepared by: Abdul Rehman**

Organization: ArrowStack Internship

Report date: 12 October 2026

## 1. Executive Summary

This report documents a network-traffic inspection exercise using Wireshark and a Python-based PCAP analysis workflow. The objective was to observe protocol activity, inspect packet metadata, and identify items that merit further validation. The supplied screenshots show traffic involving private-network addresses, DNS queries and responses, TCP/TLS 1.3 sessions, QUIC, STUN, SSDP, NAT-PMP and ICMP destination-unreachable messages. These observations describe the captured traffic; they do not, by themselves, prove that an attack or compromise occurred.

## 2. Objectives

- Open and inspect packet-capture files in Wireshark.
- Use display filters to isolate traffic such as UDP and TCP.
- Review source and destination IP addresses, protocols, packet lengths and packet details.
- Identify visible DNS activity, encrypted web sessions and network error messages.
- Use a Python workflow to support repeatable PCAP analysis and document results accurately.
- Record limitations, privacy considerations and recommendations for safe handling of packet captures.

## 3. Tools and Materials

- Wireshark – packet inspection and display filtering.
- Python 3 – scripting environment for repeatable analysis.
- Scapy / Pandas / Matplotlib – libraries referenced by the analysis workflow (verify actual use against the submitted script and requirements file).
- Input capture files supplied for this task: Network_Trafic.pcapng and Network_Trafic_Anlyzing.pcapng.
- Evidence screenshots captured on 12 October 2026.

Note: the screenshots show the Wireshark interface and the Python environment, but the available evidence does not include a complete machine-generated summary table or chart. Therefore, this report does not invent protocol percentages, total packet counts for the uploaded PCAP files, or performance measurements.

## 4. Methodology

1. **Capture/file preparation:** Use a PCAPNG file obtained from an authorized device or lab. Preserve an original copy and work on a duplicate.
2. **Open capture:** Open the capture in Wireshark and confirm that packets and protocol columns are visible.
3. **Filter traffic:** Apply the display filter udp to inspect UDP-related packets and tcp to inspect TCP traffic. A green filter bar indicates a syntactically valid display filter.

4. **Inspect packet details:** Select representative packets and review Ethernet, IPv4, TCP/UDP and application-protocol details in the packet-details pane.
5. **Interpret observations:** Describe protocol behaviour and anomalies conservatively. An unusual or failed packet is a lead for investigation, not proof of malicious activity.
6. **Python analysis:** Run the supplied analyze_pcap.py workflow in the project virtual environment, verify dependencies, and save outputs. Re-run against the final submitted capture to produce reproducible results.
7. **Evidence and reporting:** Save screenshots, record the capture filename and analysis date, and redact sensitive identifiers before publishing evidence.

## 5. Findings from the Provided Evidence

| Observation | Evidence visible | Interpretation / follow-up |
|---|---|---|
| **Private IPv4 host** | 10.141.3.52 appears as a source or destination in the screenshots. | Likely a host on the observed local/private network. Confirm the capture interface and device identity locally. |
| **DNS activity** | DNS queries and responses are visible; one example query is for auth.grammarly.com, with DNS server 192.168.230.53 shown in packet details. | DNS is expected on most networks. Domain names can reveal user activity; redact before public sharing. |
| **TCP and TLS 1.3** | TCP traffic to destination port 443 and TLSv1.3 application-data records are visible. | Consistent with encrypted web/application sessions. Encrypted payloads generally cannot be read from these packet views alone. |
| **QUIC** | QUIC packets appear in the protocol column. | QUIC commonly supports modern encrypted transport, often for web applications. Presence alone is not suspicious. |
| **STUN** | STUN Binding Success Response packets appear with XOR-MAPPED-ADDRESS values. | Often associated with NAT traversal and real-time communication. Validate against expected apps and network use. |
| **SSDP / NAT-PMP / PCP** | The screenshot shows SSDP M-SEARCH, NAT-PMP and PCP-related messages. | These can be used for local discovery or NAT/device mapping. Confirm whether the applications and router features are expected. |
| **ICMP destination unreachable** | Several ICMP messages state Destination unreachable (Port | May indicate a service/port was unavailable or a reply to UDP traffic. Correlate |

| | | |
|---|---|---|
| | unreachable). | timestamps and surrounding packets before concluding there is a problem. |
| **Capture volume/filter status** | One Wireshark view reports 132,161 packets with 706 displayed (0.5%) under the UDP filter. Another TCP view reports 101,247 packets and 100,847 displayed (99.6%). | These are screenshot status-bar values for the displayed capture/session, not verified totals for both uploaded files. Record exact file and filter before quoting them in final metrics. |

## 6. Security Assessment

Based on the supplied screenshots, the capture contains a mixture of ordinary network protocols and some messages that deserve contextual review. No confirmed malicious activity can be established from these screenshots alone. In particular, DNS, TLS, QUIC, STUN, SSDP, NAT-PMP/PCP and ICMP messages can all occur during normal device and application operation.

## 7. Recommendations

- Capture only traffic you are authorized to inspect, ideally in a dedicated lab or on your own device/network.
- Keep the original PCAPNG files private; they may expose browsing domains, local IP addresses, device details and timing information.
- Before placing screenshots or captures on a public GitHub repository, redact private IPs, public IPs, DNS names, MAC addresses, usernames and other identifiers where practical. Prefer sanitized screenshots over raw captures.
- For ICMP port-unreachable messages, inspect the packet sequence and related UDP flow before deciding whether any remediation is needed.
- Check whether SSDP, NAT-PMP and PCP are expected in the environment. Disable router/device features only after understanding the impact and obtaining authorization.
- Add a script-generated CSV/JSON summary and charts only after running the analysis against the exact capture intended for submission.
- Do not label traffic malicious solely because it uses an unfamiliar domain, a nonstandard protocol or an external IP address.

## 8. Limitations

- The findings are based on screenshots and the existence of the supplied capture files; a full decoded, reproducible analysis of each PCAPNG was not available for this report.
- The screenshot status-bar counts may refer to different capture states or files and are not combined into one total.
- No validated protocol-distribution percentages, top-talkers table, throughput calculation, or threat attribution is claimed.

- TLS application data is encrypted in the visible evidence, so the application payload cannot be inferred from these screenshots.

## 9. Conclusion

The exercise demonstrates the use of Wireshark display filters and packet-detail inspection to identify network protocols, endpoint metadata and selected network errors. The evidence shows DNS, TCP/TLS 1.3, QUIC, STUN, SSDP, NAT-PMP/PCP and ICMP activity. The appropriate conclusion is that the capture provides useful material for traffic analysis, while any security concern requires correlation with the full capture, the device's expected behaviour and additional logs. The final submission should include the actual Python script, requirements file, sanitized screenshots and generated output from a fresh run.

## 10. Reproducibility Checklist Before Submission

☐ Confirm analyze_pcap.py runs without errors in the project virtual environment.

☐ Run the script against the exact PCAPNG file included or referenced in the project.

☐ Save the terminal output and any CSV/JSON/PNG analysis outputs.

☐ Check that all counts and percentages in the report match those generated by the final run.

☐ Remove .venv, secrets, tokens and unnecessary raw capture files from the public repository.

☐ Use a README with setup, command examples, expected outputs, limitations and authorized-use statement.

☐ Review all screenshots for personal data and redact sensitive details before publishing.

## Appendix A. Evidence Screenshots

Figure A1. Wireshark UDP-filtered view showing DNS, STUN, SSDP/NAT-PMP/PCP and ICMP messages.

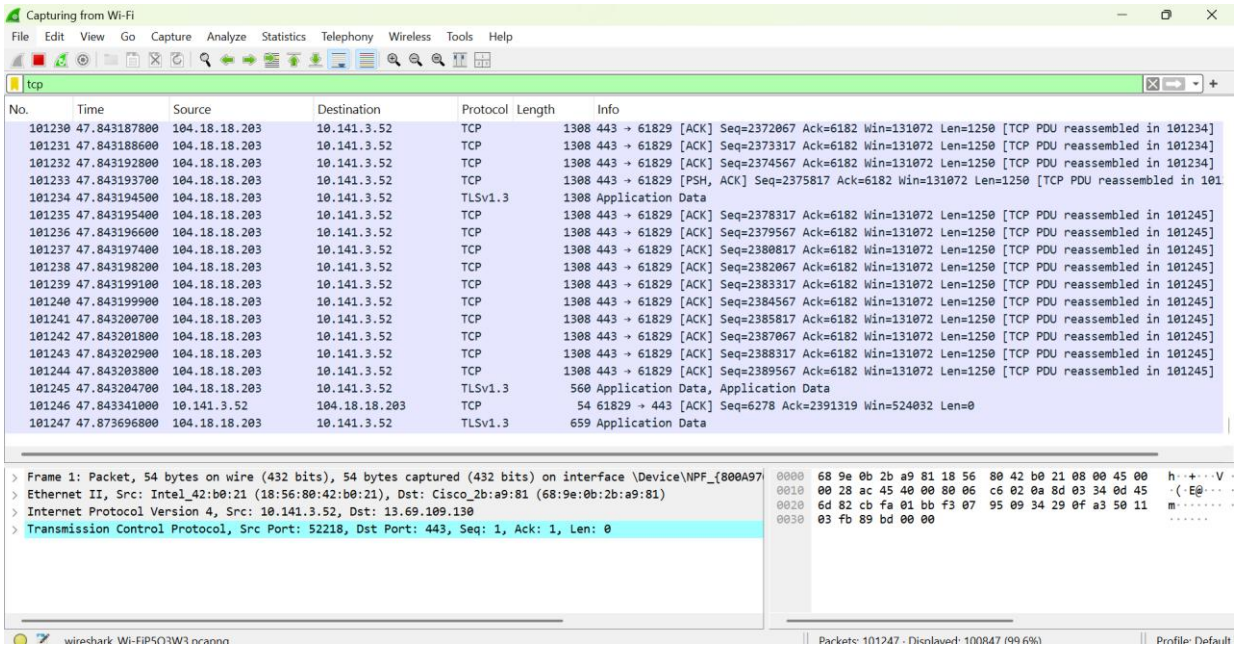



Figure A2. Wireshark capture showing UDP/QUIC and DNS traffic.

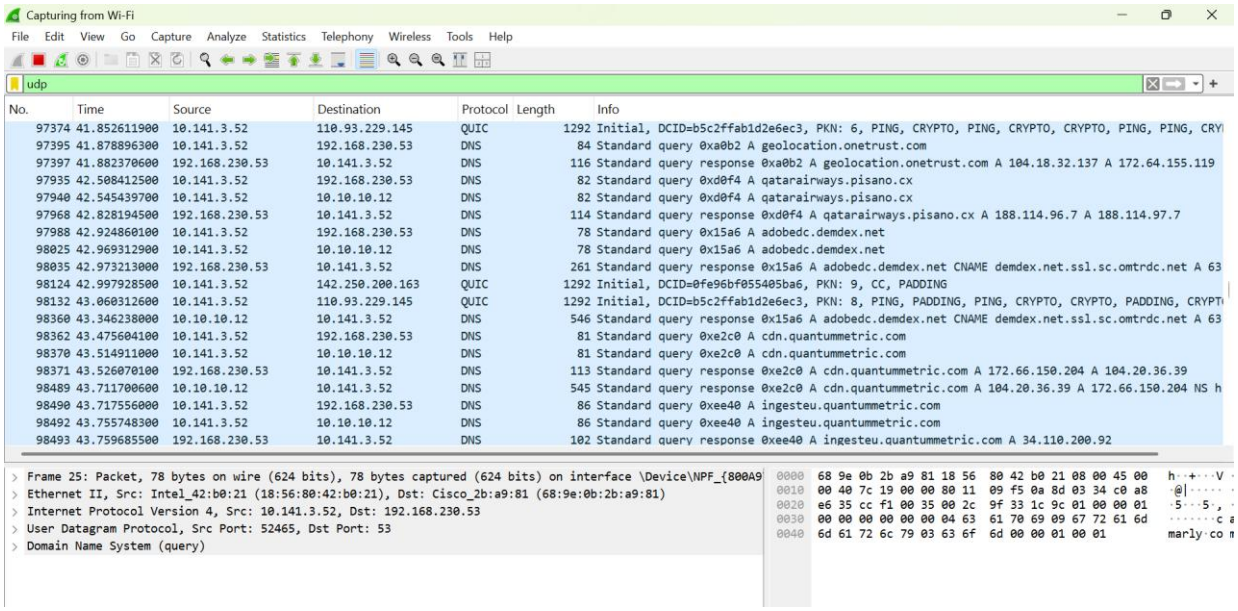



Figure A3. Wireshark TCP-filtered view showing TCP port 443 and TLS 1.3 application-data records.

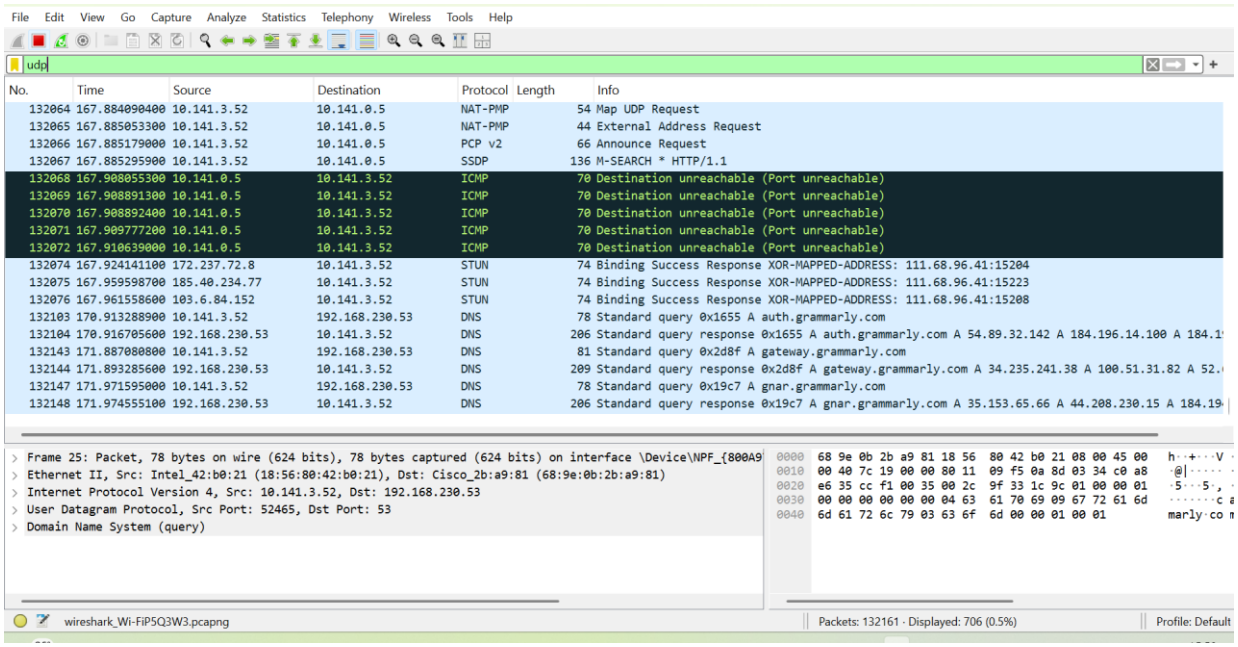



Figure A4. Python environment showing the analyze_pcap.py file and installed Pandas package information.

